

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	7
Student Name	Kavya G.
Roll No.	A24126552082

1. TITLE

Express.js Middleware – Logging Incoming Requests

2. AIM

To understand middleware in Express.js, explain how middleware works in the request-response cycle, and demonstrate middleware used for logging incoming requests.

3. OBJECTIVE

- To understand the concept of middleware in Express.js.
 - To understand how middleware works in the request-response cycle.
 - To create a custom logging middleware function.
 - To log the date, HTTP method, and requested URL.
 - To use the next() function to continue request processing.
 - To apply middleware using app.use().
-

4. THEORY

Middleware in Express.js is a function that runs between receiving a request and sending a response. It has access to the request object (req), response object (res), and the next() function. Middleware can be used for logging, authentication, validation, and request processing.

In this program, a custom logger middleware records the current date and time, HTTP method, and requested URL. The next() function passes control to the next middleware or route handler. The app.use(logger) statement applies the logging middleware to incoming requests.

5. ALGORITHM / PROCEDURE

1. Import the Express.js module using require().
2. Create an Express application using express().
3. Define a logger middleware function with req, res, and next parameters.
4. Display the current date and time, request method, and request URL.
5. Call next() to continue the request-response cycle.
6. Use app.use(logger) to apply the middleware.
7. Create the / route to display a welcome message.
8. Create the /about route to display the about page.
9. Start the server on port 3000.

10. Access the routes and observe the logged request details in the console.

6. PROGRAM

```
const express = require("express");
const app = express();

// Logging middleware
function logger(req, res, next) {
  console.log(
    new Date().toLocaleString(),
    req.method,
    req.url
  );
  next();
}

// Use middleware
app.use(logger);

// Routes
app.get("/", (req, res) => {
  res.send("Welcome to my server");
});

app.get("/about", (req, res) => {
  res.send("This is the about page");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

7. CODE EXPLANATION

Code / Function	Explanation
<code>require("express")</code>	Imports the Express.js module.
<code>express()</code>	Creates an Express application.
<code>logger(req, res, next)</code>	Defines custom middleware for logging incoming requests.
<code>new Date().toLocaleString()</code>	Gets the current date and time in readable form.
<code>req.method</code>	Displays the HTTP method used by the request.
<code>req.url</code>	Displays the URL requested by the client.
<code>next()</code>	Passes control to the next middleware or route handler.
<code>app.use(logger)</code>	Applies the logger middleware to incoming requests.
<code>app.get("/", ...)</code>	Defines the home route and sends a welcome message.
<code>app.get("/about", ...)</code>	Defines the about route and sends an about-page message.
<code>app.listen(3000)</code>	Starts the server and listens on port 3000.

8. RESULT

Thus, the Express.js middleware concept was studied successfully. A custom logging middleware was created to display the request date and time, HTTP method, and URL. The middleware was applied using `app.use()` and successfully processed requests before the route handlers.